First Year MTech (Computer Engineering) (2025 Pattern)
R25-MCO-PCC-105: Laboratory Practice I- Student Lab Manual

Index

Expt. No.	Experiment Title	Page No.	Performance Date	Submission Date	Remark	Sign
1	Exploring Set , Relations, and Functions using Programming.					
2	Simulating Probability Distributions					
3	Case-Study: Application of Divide & Conquer in Image Processing and Financial Modelling					
4	Comparative Analysis: Greedy vs. Dynamic Programming					
5	AI Face Recognition Entry Agent					
6	Clustering with K-Means and DBSCAN (Iris + Synthetic data)					
7	Descriptive Analytics & Visualization of Various Data Types					
8	Predictive Analytics & Machine Learning Models					

CERTIFICATE

This is to certify that Mr/Ms _____

Roll no- _____ Examination seat no-_____ has performed above mentioned practical's in the college.

Dr. S.K.Sonkar
Faculty Incharge

Dr.S.K.Sonkar
Head of the Department

Experiment No. 1: Exploring Sets, Relations, and Functions using Programming

Aim

To write programs that perform operations on sets, check properties of relations, and verify characteristics of functions such as injective, surjective, and bijective.

Problem Statement

1. Accept two sets **A** and **B** from the user and compute:
    - Union
    - Intersection
    - Difference
    - Cartesian Product
2. Accept a relation **R** on set **A** and check whether it is:
    - Reflexive
    - Symmetric
    - Transitive
3. Define a function from **A** to **B** and determine whether it is:
    - One-to-one (Injective)
    - Onto (Surjective)
    - Bijective

Theory

Set Operations

- **Union (A ∪ B):** Elements present in A or B.
- **Intersection (A ∩ B):** Elements common in A and B.
- **Difference (A − B):** Elements in A but not in B.
- **Cartesian Product (A × B):** Ordered pairs (a, b) where a ∈ A and b ∈ B.

Properties of Relations

A relation **R** on set **A** is a subset of A × A.

- **Reflexive:** (a, a) ∈ R for all a ∈ A
- **Symmetric:** If (a, b) ∈ R, then (b, a) ∈ R
- **Transitive:** If (a, b) ∈ R and (b, c) ∈ R, then (a, c) ∈ R

Functions and Their Properties

A function f: A → B maps each element of A to exactly one element of B.

- **Injective (One-to-one):** No two elements of A map to the same element of B.
- **Surjective (Onto):** Every element of B has at least one element in A mapped to it.
- **Bijective:** Both injective and surjective.

Algorithms

Algorithm 1: Set Operations

1. Input set A
2. Input set B
3. Compute A ∪ B
4. Compute A ∩ B
5. Compute A − B
6. Compute Cartesian Product

7. Display results

Algorithm 2: Relation Properties

1. Input set A
2. Input relation R as a set of ordered pairs
3. Check reflexive: verify (a,a) exists for all a
4. Check symmetric: for each (a,b), check (b,a)
5. Check transitive: for pairs (a,b) and (b,c), check (a,c)
6. Display results

Algorithm 3: Check Function Properties

1. Input set A
2. Input set B
3. Input mapping f as ordered pairs
4. Check injective → values in B are unique
5. Check surjective → all elements of B appear
6. If both true → bijective
7. Display results

Python Program

```
# ----------------------------
# Part 1: Set Operations
# ----------------------------
A = set(map(int, input("Enter elements of set A: ").split()))
B = set(map(int, input("Enter elements of set B: ").split()))
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference (A - B):", A - B)
print("Cartesian Product:", {(a, b) for a in A for b in B})
# ----------------------------
# Part 2: Relation Properties
# ----------------------------
n = int(input("Enter number of ordered pairs in relation R: "))
R = set()

print("Enter pairs (a b):")
for _ in range(n):
    a, b = map(int, input().split())
    R.add((a, b))

# Reflexive
reflexive = all((a, a) in R for a in A)
# Symmetric
symmetric = all((b, a) in R for (a, b) in R)
# Transitive
transitive = True
```

```
for (a, b) in R:
    for (c, d) in R:
        if b == c and (a, d) not in R:
            transitive = False
print("Reflexive:", reflexive)
print("Symmetric:", symmetric)
print("Transitive:", transitive)
# ----------------------------
# Part 3: Function Properties
# ----------------------------
m = int(input("Enter number of function mappings f: "))
f = {}
print("Enter mappings (a b) meaning a -> b:")
for _ in range(m):
    a, b = map(int, input().split())
    f[a] = b

# Injective
injective = len(set(f.values())) == len(f.values())
# Surjective
surjective = set(f.values()) == B
bijective = injective and surjective
print("Injective:", injective)
print("Surjective:", surjective)
print("Bijective:", bijective)
```

Sample Output

```
Enter elements of set A: 1 2 3
Enter elements of set B: 3 4 5
Union: {1, 2, 3, 4, 5}
Intersection: {3}
Difference (A - B): {1, 2}
Cartesian Product: {(1,3),(1,4),(1,5),...}
Enter number of ordered pairs in R: 3
Enter pairs:
1 1
2 2
1 2
Reflexive: False
Symmetric: False
Transitive: True
Enter number of function mappings: 3
Enter pair:
1 3
```

2 4
3 5
Injective: True
Surjective: False
Bijective: False

Conclusion

In this assignment, operations on sets were successfully performed, relations were analyzed using mathematical properties, and functions were evaluated for injective, surjective, and bijective characteristics using programming.

Questions

1. What is the difference between relation and function?
2. Define reflexive and symmetric relations with examples.
3. What is meant by injective and surjective mapping?
4. Explain Cartesian Product with an example.
5. Can a relation be transitive but not reflexive? Explain.

Experiment No 2: Simulating Probability Distributions

Aim

To write programs for simulating coin tosses, dice rolls, and to generate Normal and Poisson probability distributions using Python (NumPy/Matplotlib).

Problem Statement

1. Simulate probability experiments:
    - Toss a coin **1000 times**, count Heads and Tails, and plot their frequencies.
    - Roll a dice **1000 times** and compute the probability distribution of outcomes (1–6).
2. Generate and plot:
    - **Normal Distribution** for given mean (μ) and standard deviation (σ).
    - **Poisson Distribution** for a given parameter λ (lambda).

Theory

Coin Toss Simulation

A coin has two outcomes: **Head (H)** or **Tail (T)**, each with probability 0.5 in an ideal scenario.

Simulating random outcomes allows us to observe real-world deviations from ideal probability.

Dice Roll Simulation

A fair dice has outcomes {1, 2, 3, 4, 5, 6}.

The theoretical probability $P(x) = 1/6 \approx 0.1667$ for each outcome.

Simulated frequencies approximate these values as sample size increases.

Normal Distribution

A continuous probability distribution characterized by:

- Mean μ
- Standard deviation σ

Formula:

$$f(x)=\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Used in real-world models such as measurement variations, heights, noise patterns.

Poisson Distribution

A discrete distribution for modelling the count of events occurring in an interval, given by λ.

Formula:

$$P(X=k)=\frac{e^{-λ} λ^k}{k!}$$

Used in call arrivals, packet drops, traffic flow, etc.

Algorithms

Algorithm 1: Coin Toss Simulation

1. Set n = 1000.
2. For each trial, generate random choice (Head/Tail).
3. Count frequencies of H and T.
4. Plot a bar graph.

Algorithm 2: Dice Roll Probability Distribution

1. Set n = 1000.
2. Generate 1000 random integers in range 1–6.
3. Count occurrences of each number.
4. Compute probability = count / n.
5. Plot bar graph.

Algorithm 3: Normal Distribution

1. Input μ and σ.
2. Generate 1000 random values using NumPy's normal() function.
3. Plot histogram and PDF curve.

Algorithm 4: Poisson Distribution

1. Input λ.
2. Generate 1000 values using NumPy's poisson() function.
3. Plot histogram.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------
# Part 1: Coin Toss Simulation
# ------------------------------------
n = 1000
coin = np.random.choice(["H", "T"], n)
heads = np.sum(coin == "H")
tails = np.sum(coin == "T")

plt.bar(["Heads", "Tails"], [heads, tails])
plt.title("Coin Toss Frequency (1000 trials)")
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.show()

# ------------------------------------
```

```
# Part 2: Dice Roll Simulation
# ------------------------------------
dice = np.random.randint(1, 7, n)
values, counts = np.unique(dice, return_counts=True)
probabilities = counts / n

plt.bar(values, counts)
plt.title("Dice Roll Distribution (1000 trials)")
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.show()

print("Dice Probabilities:", dict(zip(values, probabilities)))

# ------------------------------------
# Part 3: Normal Distribution
# ------------------------------------
mu = 0
sigma = 1
normal_data = np.random.normal(mu, sigma, n)

plt.hist(normal_data, bins=30, density=True)
plt.title("Normal Distribution (μ=0, σ=1)")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()

# ------------------------------------
# Part 4: Poisson Distribution
# ------------------------------------
lam = 4
poisson_data = np.random.poisson(lam, n)

plt.hist(poisson_data, bins=range(0, 15), align='left', rwidth=0.8)
plt.title("Poisson Distribution (λ=4)")
plt.xlabel("k")
plt.ylabel("Frequency")
plt.show()
```

Sample Output

Coin Toss:
Heads = 510
Tails = 490

Dice Probabilities:
{1: 0.171, 2: 0.158, 3: 0.164, 4: 0.170, 5: 0.166, 6: 0.171}
(Graphs will display coin bar chart, dice distribution, normal curve, and Poisson histogram.)

Conclusion

This assignment successfully demonstrated how probability distributions can be simulated using Python. Through repeated random trials, the observed frequencies approximate theoretical probabilities. Normal and Poisson distributions were generated and visualized, enhancing understanding of statistical behaviour.

Questions

1. What is the difference between theoretical and experimental probability?
2. Why does increasing the number of trials give more accurate results?
3. Explain Normal and Poisson distributions with real-world applications.
4. How does NumPy help in simulating random events?
5. What is λ in Poisson distribution and how does it affect shape?

Experiment No 3: Case Study: Application of Divide & Conquer in Image Processing and Financial Modelling

Introduction

Divide & Conquer (D&C) is a problem-solving paradigm that breaks a complex problem into smaller subproblems, solves each independently, and combines the results. It is widely used in fields requiring speed, scalability, and parallel execution. Two major real-world domains that leverage D&C heavily are **Image Processing** and **Financial Modeling**.

Part A: Divide & Conquer in Image Processing

Background

Modern image processing involves tasks like compression, segmentation, edge detection, filtering, and pattern recognition. Because images are typically large matrices of pixels, operations on them can be computationally expensive.
D&C enables breaking down large images into blocks or segments so that operations become:

- Faster
- Parallelizable
- Memory efficient

Real-World Use Case: JPEG Image Compression

Problem

Efficiently compress large images while balancing quality and file size.

How Divide & Conquer Is Used

JPEG compression uses D&C by splitting the image into **8×8 blocks**. Each block is processed individually:

1. **Divide**
    - The image is divided into multiple fixed-size sub-images (blocks).
    - Each block is treated as an independent subproblem.
2. **Conquer**
    - Apply Discrete Cosine Transform (DCT) on each block.
    - Quantize the transformed coefficients.
    - Perform entropy encoding (Huffman coding).
3. **Combine**
    - Combine the compressed blocks back into one final file.

Benefits

- Localizing compression reduces computational load.
- Blocks enable parallel processing (multi-core CPUs, GPUs).
- Adjustable compression allows optimization based on required quality.

Real-World Use Case: Image Segmentation in Medical Imaging (CT/MRI)
Problem
Segmenting organs, tissues, or tumors from high-resolution scans.
D&C Solution

1. **Divide**
    - Large medical images (often 4K+ resolution) are split into tiles.
    - Each tile undergoes noise removal, thresholding, or clustering.
2. **Conquer**
    - Classification algorithms like k-means, region growing, or CNN filters are applied to each tile.
3. **Combine**
    - Segmented tiles are stitched together.
    - Boundary refinement corrects edges between tiles.

Benefits

- Faster segmentation even with large datasets.
- Reduces RAM usage.
- Parallel tile processing shortens output time—critical in medical diagnosis.

5. Real-World Use Case: Fast Fourier Transform (FFT) in Image Filtering
FFT is inherently a Divide & Conquer algorithm.
How it is used:

- FFT breaks down an N-point signal into smaller N/2 segments recursively.
- Used for image filtering, watermarking, and frequency domain analysis.

Advantages:

- Reduces computation from **$O(n^2)$** to **$O(n \log n)$**.
- Enables real-time video processing applications.

Part B: Divide & Conquer in Financial Modelling
Background
Financial markets deal with huge datasets and time-sensitive decision-making. Tasks such as option pricing, portfolio optimization, or risk analysis often require D&C strategies to improve performance.

Real-World Use Case: Monte Carlo Simulation for Option Pricing
Problem
Evaluating complex derivatives (e.g., European options) requires simulating millions of price paths.
D&C Approach

1. **Divide**
    - Split simulation work into multiple independent subsets (batches of price paths).
    - Each batch can be assigned to a CPU/GPU core.
2. **Conquer**

    - Run stochastic price evolution for each subset.
    - Compute expected payoff for each batch.
3. **Combine**
    - Average all payoffs to calculate the final option price.

Results

- Faster execution using parallelism.
- Scalable to large datasets.
- Used heavily by banks (Goldman Sachs, JP Morgan) for quant analysis.

Real-World Use Case: Portfolio Optimization using D&C

Modern portfolio theory involves minimizing risk and maximizing return, often requiring optimization over thousands of assets.

How D&C is applied:

1. **Divide**
    - Segment the portfolio into sub-portfolios (e.g., by sector or geography).
2. **Conquer**
    - Optimize each sub-portfolio independently (mean-variance calculation, covariance estimation).
3. **Combine**
    - Merge optimized sub-portfolios and perform global calibration.

Benefits

- Reduced computational complexity.
- Improved scalability for Big Data financial systems.
- Supports real-time risk monitoring.

Real-World Use Case: Binary Search for High-Frequency Trading (HFT)

Problem

HFT platforms need to search for price changes in microseconds.

Solution Using Divide & Conquer

Binary Search reduces lookup time to **O(log n)**.

Used for:

- Price matching
- Order book navigation
- Arbitrage detection

Impact

- Faster decision-making.
- Higher profits with minimized latency.

Key Advantages of Using Divide & Conquer

Benefit	**Explanation**
Speed	Allows breaking large tasks into smaller, faster computations.
Parallelism	Independent subtasks run on multiple processors/cores.

Benefit	Explanation
Scalability	Handles large datasets (image pixels, financial time-series).
Reduced Memory Load	Only small segments are loaded at a time.
Modularity	Easier debugging and future enhancements.

Conclusion

Divide & Conquer is a fundamental strategy in both engineering and financial industries.
In **Image Processing**, it enables fast compression, segmentation, and filtering using block-based processing and FFT.
In **Financial Modeling**, it accelerates simulations, optimizations, and real-time trading decisions.
Its efficiency, simplicity, and compatibility with modern parallel hardware (multi-core CPUs, GPUs, cloud systems) make it indispensable for processing large-scale, real-world computational problems.

Questions

1. Explain the Divide & Conquer steps used in JPEG compression.
2. Why is D&C suitable for Monte Carlo simulations?
3. How does FFT leverage Divide & Conquer principles?
4. Compare D&C with dynamic programming in terms of use cases.
5. Provide one additional real-world application of D&C not discussed above.

Experiment No 4: Comparative Analysis: Greedy vs. Dynamic Programming

Introduction

Greedy Algorithms and Dynamic Programming (DP) are two widely used algorithmic strategies for solving optimization problems. Both break problems into smaller subproblems but differ fundamentally in how they choose solutions.

Greedy Algorithm

Definition

A greedy algorithm builds a solution step-by-step by choosing the **locally optimal choice** at each stage, hoping it leads to a global optimum.

Characteristics

- Makes immediate, short-term optimal choices
- No backtracking
- Fast and easy to implement
- Works only when the problem exhibits **greedy choice property** and **optimal substructure**

Advantages

- Very efficient (often O(n log n) or O(n))
- Simple logic

Disadvantages

- May fail to produce optimal solution for complex problems
- Requires proof that greedy choices always lead to optimal results

3. Dynamic Programming (DP)

Definition

DP solves problems by breaking them into overlapping subproblems, storing intermediate results in a **table**, and combining them to build optimal solutions.

Characteristics

- Uses **memoization** (top-down) or **tabulation** (bottom-up)
- Ensures globally optimal solution
- Works when the problem has **optimal substructure** and **overlapping subproblems**

Advantages

- Guarantees optimal solution
- Handles complex constraints

Disadvantages

- Higher time and memory usage than greedy
- More complex to implement

Key Differences Between Greedy and DP

Feature	Greedy Algorithm	Dynamic Programming
Strategy	Takes local best choice	Examines all possibilities

Feature	Greedy Algorithm	Dynamic Programming
Optimality	Not always optimal	Always optimal
Subproblems	Independent	Overlapping
Memory Use	Low	High (stores tables)
Speed	Fast	Slower
Complexity	Simple logic	More complex
Backtracking	No	Yes (implicitly through DP table)
Suitable Problems	Huffman coding, activity selection	Knapsack, shortest paths (Bellman–Ford, Floyd–Warshall)

Example 1: 0/1 Knapsack Problem

Problem:

Given items with weight and value, maximize value within weight capacity **W**.

Greedy Approach (Fails)

Greedy strategy picks items with highest **value/weight ratio** first.

Example Data:

Item	Weight	Value	Ratio
A	10	60	6
B	20	90	4.5
C	30	120	4

Capacity **W = 50**

Greedy picks:

1. Item A (weight 10)
2. Item B (weight 20)
3. Cannot pick C (exceeds capacity)

Total value = **150**

Dynamic Programming (Optimal Solution)

DP evaluates all combinations through table building.

Optimal Selection:

- Items **B + C** → weight = 50, value = **210**
- Better than Greedy's **150**

Conclusion for Example 1

- Greedy fails because local best (ratio) does not guarantee global best.
- DP finds optimal solution.

Example 2: Shortest Path Problem

Case 1: Greedy Works — Dijkstra's Algorithm
When graph has:

- **non-negative edge weights**

Dijkstra takes the **greedy shortest local distance** at each step and obtains globally optimal shortest paths.
Example Graph
(source → A: 4, source → B: 1, B → A: 2)
Greedy step-by-step selection finds correct shortest paths.
✓ Works perfectly because the graph satisfies constraints.

Case 2: Greedy Fails — When Graph Has Negative Edges
Example
Edges:

- A → B = 4
- A → C = 2
- C → B = –5

Greedy (Dijkstra) picks:

- A → C first (2)
  Then believes C → B path cannot be better than A → B = 4 initially.

But actual shortest path:

- A → C → B = 2 + (–5) = **–3**

Dijkstra fails due to negative edges.

Dynamic Programming Solution — Bellman–Ford Algorithm
Bellman–Ford evaluates all edges **V–1 times** and finds correct shortest paths even with negative weights.

Summary of Both Examples

Problem	Greedy	DP	Result
0/1 Knapsack	Fails	Works	DP is optimal
Shortest Path	Works with non-negative weights	Works in all cases	DP more general

Conclusion
Greedy and Dynamic Programming are both powerful approaches, but their applicability differs:

- Use **Greedy** when the problem structure guarantees that locally optimal choices lead to globally optimal results.
- Use **Dynamic Programming** when problems involve overlapping subproblems and ensuring optimal results is crucial.

DP is more robust but computationally heavier; Greedy is faster but requires strict conditions for correctness.

Questions

1. Define greedy choice property and optimal substructure.
2. Why does Greedy fail for 0/1 Knapsack but succeed for Fractional Knapsack?
3. Compare Dijkstra and Bellman–Ford algorithms.
4. Give one real-world example for Greedy and one for DP.
5. Explain how memorization improves efficiency in DP.

Experiment No 5: AI Face Recognition Entry Agent

Aim

Build a real-time face recognition system that detects faces from a webcam and recognizes known people using a CNN-based embedding extractor and a classifier. The system grants or denies entry based on whether the detected face matches a known person in the pre-trained database.

Learning Outcomes

- Prepare a dataset of labeled faces and perform preprocessing (detection, alignment, augmentation).
- Use transfer learning (MobileNetV2) as a feature extractor (embedding generator).
- Train a classifier (SVM / KNN / RandomForest) on embeddings for identity recognition.
- Implement a real-time face detection + recognition pipeline and decision logic for entry control.
- Understand evaluation metrics and ethical/privacy requirements.

System Overview (High-level)

1. **Data collection**: Collect several images per person and organize folders dataset/<person_name>/img1.jpg ....
2. **Preprocessing**: Detect faces in images, crop, resize (160×160 or 224×224), optionally align.
3. **Embedding model**: Use a pre-trained CNN backbone (MobileNetV2) with include_top=False + global pooling to create fixed-size embeddings.
4. **Classifier training**: Train an SVM (or KNN) on these embeddings + labels; save classifier and label encoder.
5. **Realtime**:
    - Detect faces on webcam frames (OpenCV DNN or Haar cascade).
    - Extract embedding for each face.
    - Classify embedding (predict identity & probability).
    - If confidence above threshold → grant entry; otherwise deny.
6. **Logging & Access Control**: Log recognized entries with timestamps; optionally trigger door/gpio.

Requirements

(Use a Python virtual env)

- Python 3.8+
- Packages (pip):

numpy
opencv-python
tensorflow # or tensorflow-cpu
scikit-learn
imutils
pillow

tqdm
Optional but recommended:
mtcnn # for better face detection/alignment
dlib # if you want robust alignment (hard to install on Windows)
Create requirements.txt accordingly.

Dataset Format
Folder structure:
dataset/
  Alice/
    alice1.jpg
    alice2.jpg
  Bob/
    bob1.jpg
    bob2.jpg
  UnknownPerson/ # optional negative examples
Aim for 20–100 images per person (more is better). Vary pose, lighting, expression.

Design Choices & Notes

- **Embedding model**: MobileNetV2 (pretrained on ImageNet) used as feature extractor (transfer learning). Using include_top=False + GlobalAveragePooling2D yields an embedding vector (e.g., 1280-d).
- **Classifier**: SVM with probability estimates (via probability=True) gives robust decision boundaries and probabilities. KNN is an alternative (distance-based).
- **Face detector**: OpenCV DNN face detector (res10_300x300_ssd_iter_140000.caffemodel) is reliable; MTCNN or dlib can be used for better alignment.
- **Threshold**: Use probability threshold (e.g., 0.6) OR distance threshold (cosine distance < 0.4) depending on classifier. Tune using validation set.
- **Safety & Ethics**: Only use with consent. Log minimal data and secure storage.

Pipeline Implementation — Files

1. embedding_model.py — builds/returns the Keras embedding extractor.
2. prepare_dataset.py — loads dataset, detects faces, preprocesses, produces array of face images and labels.
3. train_classifier.py — extracts embeddings with embedding model, trains classifier (SVM), saves classifier.pkl and label_encoder.pkl.
4. recognize_realtime.py — realtime application for detection and recognition and grants/denies entry.

Below I provide full code for the main scripts.

1) embedding_model.py
embedding_model.py

```
import tensorflow as tf
from tensorflow.keras import layers, Model

def get_embedding_model(input_shape=(224, 224, 3), backbone='MobileNetV2'):
    """
    Returns a Keras model that maps an input face image to a fixed-length embedding.
    """
    if backbone == 'MobileNetV2':
        base = tf.keras.applications.MobileNetV2(
            input_shape=input_shape, include_top=False, weights='imagenet')
        x = base.output
        x = layers.GlobalAveragePooling2D()(x)
        # Optional dense projection to reduce embedding size
        x = layers.Dense(256, activation='relu')(x)    # embedding size 256
        x = layers.BatchNormalization()(x)
        # L2-normalize embedding (useful for distance metrics)
        embedding = tf.keras.layers.Lambda(lambda t: tf.math.l2_normalize(t, axis=1))(x)
        model = Model(inputs=base.input, outputs=embedding, name='embedding_model')
        return model
    else:
        raise ValueError("Unsupported backbone")
```

2) prepare_dataset.py

This script detects faces in images (OpenCV DNN) and prepares preprocessed face images and labels.

```
# prepare_dataset.py
import os
import cv2
import numpy as np
from tqdm import tqdm

# Path to OpenCV DNN face detector (deploy prototxt and caffemodel).
# You need to download these two files and place them in 'models/' folder:
# - deploy.prototxt.txt
# - res10_300x300_ssd_iter_140000.caffemodel
FACE_PROTO = "models/deploy.prototxt.txt"
FACE_MODEL = "models/res10_300x300_ssd_iter_140000.caffemodel"

def load_opencv_dnn_detector(proto=FACE_PROTO, model=FACE_MODEL):
    net = cv2.dnn.readNetFromCaffe(proto, model)
    return net

def detect_face_dnn(net, image, conf_threshold=0.6):
```

```
    h, w = image.shape[:2]
    blob = cv2.dnn.blobFromImage(cv2.resize(image, (300, 300)),
                                 1.0, (300, 300),
                                 (104.0, 177.0, 123.0))
    net.setInput(blob)
    detections = net.forward()
    boxes = []
    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]
        if confidence > conf_threshold:
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (x1, y1, x2, y2) = box.astype("int")
            # clamp
            x1, y1 = max(0, x1), max(0, y1)
            x2, y2 = min(w - 1, x2), min(h - 1, y2)
            boxes.append((x1, y1, x2, y2))
    return boxes

def preprocess_face(face_img, target_size=(224,224)):
    face = cv2.resize(face_img, target_size)
    face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
    face = face.astype("float32")
    face = face / 255.0
    return face

def prepare_dataset(dataset_dir='dataset', img_size=(224,224),
output_npz="faces_labels.npz"):
    net = load_opencv_dnn_detector()
    X = []
    y = []
    people = sorted(os.listdir(dataset_dir))
    print("Found people:", people)
    for person in people:
        person_dir = os.path.join(dataset_dir, person)
        if not os.path.isdir(person_dir):
            continue
        for fname in tqdm(os.listdir(person_dir), desc=f"Processing {person}"):
            path = os.path.join(person_dir, fname)
            img = cv2.imread(path)
            if img is None:
                continue
            boxes = detect_face_dnn(net, img, conf_threshold=0.6)
            if len(boxes) == 0:
```

```
            continue
        # choose largest box
        x1, y1, x2, y2 = max(boxes, key=lambda b: (b[2]-b[0])*(b[3]-b[1]))
        face = img[y1:y2, x1:x2]
        face_proc = preprocess_face(face, target_size=img_size)
        X.append(face_proc)
        y.append(person)
  X = np.array(X)
  y = np.array(y)
  np.savez_compressed(output_npz, X=X, y=y)
  print(f"Saved {len(X)} faces to {output_npz}")

if __name__ == '__main__':
  prepare_dataset()
```

Run this after populating dataset/. It saves faces_labels.npz.

3) train_classifier.py

Extract embeddings using the embedding model, train an SVM classifier, and save models.

```
# train_classifier.py
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC
import joblib
from embedding_model import get_embedding_model

def load_faces_labels(npz_file='faces_labels.npz'):
  data = np.load(npz_file)
  return data['X'], data['y']

def extract_embeddings(embedding_model, faces_batch):
  # embedding_model expects input scaled [0,1] and target size (224,224,3)
  embeddings = embedding_model.predict(faces_batch, batch_size=32, verbose=1)
  return embeddings

def train(npz_file='faces_labels.npz', embedding_size=256):
  X, y = load_faces_labels(npz_file)
  print("Faces shape:", X.shape)
  model = get_embedding_model(input_shape=(224,224,3))
  embeddings = extract_embeddings(model, X)  # shape (N, embedding_size)
  print("Embeddings shape:", embeddings.shape)

  le = LabelEncoder()
  y_enc = le.fit_transform(y)
```

```
    # Train SVM classifier with probability estimates
    clf = SVC(kernel='linear', probability=True)
    clf.fit(embeddings, y_enc)

    # Save models
    joblib.dump(clf, 'classifier.pkl')
    joblib.dump(le, 'label_encoder.pkl')
    print("Saved classifier.pkl and label_encoder.pkl")

if __name__ == '__main__':
    train()
```

This generates classifier.pkl and label_encoder.pkl.

4) recognize_realtime.py

Realtime face detection + recognition with decision logic.

```
# recognize_realtime.py
import cv2
import numpy as np
import joblib
import time
from embedding_model import get_embedding_model
from sklearn.preprocessing import normalize

# Face detector files (same as prepare_dataset)
FACE_PROTO = "models/deploy.prototxt.txt"
FACE_MODEL = "models/res10_300x300_ssd_iter_140000.caffemodel"

CONF_THRESHOLD = 0.6
PROB_THRESHOLD = 0.6   # classifier probability threshold for acceptance

def load_face_detector():
    net = cv2.dnn.readNetFromCaffe(FACE_PROTO, FACE_MODEL)
    return net

def detect_faces(net, frame, conf_threshold=CONF_THRESHOLD):
    h, w = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)),
                                 1.0, (300, 300),
                                 (104.0, 177.0, 123.0))
    net.setInput(blob)
    detections = net.forward()
    boxes = []
    probs = []
```

```
    for i in range(detections.shape[2]):
        confidence = float(detections[0, 0, i, 2])
        if confidence > conf_threshold:
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (x1, y1, x2, y2) = box.astype("int")
            x1, y1 = max(0, x1), max(0, y1)
            x2, y2 = min(w - 1, x2), min(h - 1, y2)
            boxes.append((x1, y1, x2, y2))
            probs.append(confidence)
    return boxes, probs

def preprocess_face(face, target_size=(224,224)):
    face = cv2.resize(face, target_size)
    face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
    face = face.astype("float32") / 255.0
    face = np.expand_dims(face, axis=0)
    return face

def main(video_src=0):
    print("Loading models...")
    detector = load_face_detector()
    clf = joblib.load('classifier.pkl')
    le = joblib.load('label_encoder.pkl')
    embed_model = get_embedding_model(input_shape=(224,224,3))
    # Warm up
    # create video capture
    cap = cv2.VideoCapture(video_src)
    if not cap.isOpened():
        print("Cannot open camera")
        return

    print("Starting video stream. Press 'q' to quit.")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        boxes, confidences = detect_faces(detector, frame)
        for i, box in enumerate(boxes):
            x1, y1, x2, y2 = box
            face = frame[y1:y2, x1:x2]
            if face.shape[0] < 20 or face.shape[1] < 20:
                continue
            face_proc = preprocess_face(face)
```

```
            emb = embed_model.predict(face_proc)  # shape (1, embed_dim)
            # Get classifier prediction
            probs = clf.predict_proba(emb)  # shape (1, n_classes)
            best_idx = np.argmax(probs)
            best_prob = probs[0, best_idx]
            name = le.inverse_transform([best_idx])[0]

            if best_prob >= PROB_THRESHOLD:
                label = f"{name}: {best_prob:.2f} (GRANTED)"
                color = (0, 255, 0)
                # TODO: trigger door unlock mechanism here (GPIO / API)
            else:
                label = f"Unknown: {best_prob:.2f} (DENIED)"
                color = (0, 0, 255)
            # Draw box and label
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.putText(frame, label, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

            # Logging
            timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
            print(f"[{timestamp}] {label}")

        cv2.imshow("Face Recognition Entry Agent", frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == '__main__':
    main(0)
```

Notes:

- Replace # TODO: trigger door unlock mechanism here with actual integration code (GPIO toggle, REST API call to door controller, relay control, etc.).
- Save logs to a file or database for audit.

Evaluation & Tuning

1. **Validation set**: Keep a held-out validation set with images per person not used in training to tune PROB_THRESHOLD.
2. **Metrics**: Accuracy, precision/recall per identity, confusion matrix, FAR (false accept rate), FRR (false reject rate).

3. **Distance-based option**: Instead of classifier, compute mean embedding per person and use cosine distance; accept if distance < threshold (common for FaceNet-style pipelines).
    - Cosine distance: 1 - dot(u, v) where embeddings are L2-normalized.
4. **Data augmentation**: Add variations (brightness, rotation) to make the classifier robust.

Performance & Deployment Considerations

- **Speed**: Embedding extraction + SVM prediction for one face should be ~20–60 ms on CPU depending on model. Use GPU for faster throughput.
- **Memory**: Embeddings are small; classifier is small.
- **Scalability**: For hundreds/thousands identities, consider using a distance search index (FAISS) or a more scalable classifier.
- **Security**: Store classifier and embeddings securely. Use TLS for networked devices.

Potential Improvements / Variations

- Use **FaceNet** or **ArcFace** pretrained models for higher-quality embeddings (better recognition accuracy).
- Use **MTCNN** for more accurate face detection and alignment.
- Train with **triplet loss** / **ArcFace loss** for end-to-end better embeddings (requires more advanced training).
- Use **intelligent retry/fallback**: if probability close to threshold, request multi-frame confirmation.

Ethical & Legal Considerations (must include in report)

- Only register / store faces with explicit consent from persons.
- Log who accessed the system, and implement retention policies (how long logs/images are stored).
- Avoid misuse: face recognition has accuracy differences across demographics—validate fairness and document limitations.
- Follow local laws and institutional policy; inform visitors they are being recognized.

Sample Commands / Run Order

1. Create and populate dataset: dataset/<person_name>/images...
2. Prepare faces:

```
python prepare_dataset.py
# produces faces_labels.npz
```

3. Train classifier:

```
python train_classifier.py
# produces classifier.pkl & label_encoder.pkl
```

4. Run realtime agent:

```
python recognize_realtime.py
```

Troubleshooting Tips

- If face detector misses faces: lower CONF_THRESHOLD or try MTCNN.
- If misclassifications are high: add more images per person and more variation.
- If slow: use GPU TensorFlow or use a smaller backbone (MobileNetV2 lite) and reduce input size to 160×160.
- To handle multiple faces per frame, ensure cropping and embedding extraction are batched if speed is critical.

Questions:

Q1. Explain the complete workflow of a CNN-based Face Recognition Entry System.

Q2. Compare traditional image processing–based face recognition with CNN-based deep learning methods.

Experiment No 6: Clustering with K-Means and DBSCAN (Iris + Synthetic data)
Aim: Apply K-Means and DBSCAN clustering on a multidimensional dataset (Iris and synthetic), visualize clusters, and compare cluster quality using internal metrics.

Theory
K-Means: partitions data into k clusters by minimizing within-cluster variance. Works best for roughly spherical clusters and when k is known. Sensitive to initialization and scale.

- **DBSCAN**: density-based clustering that finds clusters of arbitrary shape, requires eps (neighborhood radius) and min_samples. Can find noise/outliers; good for uneven cluster sizes/density.
- **PCA**: Principal Component Analysis reduces dimensionality while retaining most variance — used for 2-D visualization.

Algorithms (summary)

1. Load dataset (Iris or synthetic via make_blobs / make_moons).
2. Standardize features (zero mean, unit variance).
3. Fit K-Means (choose k e.g., 3 for Iris).
4. Fit DBSCAN (grid-search eps & min_samples if needed).
5. Compute silhouette score & Davies-Bouldin index to compare clusters.
6. Reduce to 2D via PCA and plot cluster assignments, centroids (K-Means), and noise points (DBSCAN).

Full Python Code (run locally)

```
# clustering_lab.py
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, davies_bouldin_score
import pandas as pd

sns.set(style="whitegrid", context="notebook")

def load_datasets():
    iris = datasets.load_iris()
    X_iris = iris.data
    y_iris = iris.target
    # Synthetic datasets
```

```
    X_blobs, y_blobs = make_blobs(n_samples=500, centers=4, cluster_std=0.60,
random_state=42)
    X_moons, y_moons = make_moons(n_samples=500, noise=0.08, random_state=42)
    return (X_iris, y_iris, 'Iris (4D)'), (X_blobs, y_blobs, 'Blobs (2D)'), (X_moons, y_moons,
'Moons (2D)')

def standardize(X):
    scaler = StandardScaler()
    return scaler.fit_transform(X), scaler

def run_kmeans(X, k, random_state=42):
    kmeans = KMeans(n_clusters=k, random_state=random_state, n_init=10)
    labels = kmeans.fit_predict(X)
    centers = kmeans.cluster_centers_
    return labels, centers, kmeans

def run_dbscan(X, eps=0.5, min_samples=5):
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X)
    return labels, db

def evaluate_clusters(X, labels, name=""):
    unique_labels = np.unique(labels)
    n_clusters = len(unique_labels[unique_labels != -1])  # exclude noise label -1
    if n_clusters <= 1:
        sil = -1
        db = np.nan
    else:
        sil = silhouette_score(X, labels)
        db = davies_bouldin_score(X, labels)
    print(f"{name} -> clusters: {n_clusters}, silhouette: {sil:.4f}, davies-bouldin: {db:.4f}")
    return n_clusters, sil, db

def plot_2d(X2, labels, title="Clusters", centers2=None, show_centers=True):
    df = pd.DataFrame(X2, columns=['PC1','PC2'])
    df['cluster'] = labels.astype(str)
    plt.figure(figsize=(7,5))
    palette = sns.color_palette("husl", len(np.unique(labels)))
    # mark noise as black if present
    if -1 in labels:
        # create palette mapping but ensure -1 is black
        unique = sorted(np.unique(labels))
```

```
        palette_map = {lab: ("#000000" if lab == -1 else palette[i % len(palette)]) for i, lab in
enumerate(unique)}
        sns.scatterplot(x='PC1', y='PC2', hue='cluster', data=df,
                        palette=palette_map, legend='full', s=40)
    else:
        sns.scatterplot(x='PC1', y='PC2', hue='cluster', data=df, palette="husl", legend='full',
s=40)
    if centers2 is not None and show_centers:
        plt.scatter(centers2[:,0], centers2[:,1], c='red', s=200, marker='X', edgecolor='k',
label='centroids')
    plt.title(title)
    plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
    plt.tight_layout()
    plt.show()

def pca_transform(X, n_components=2):
    pca = PCA(n_components=n_components)
    X2 = pca.fit_transform(X)
    return X2, pca

def main():
    datasets_list = load_datasets()
    for X, y_true, name in datasets_list:
        print("\n=== Dataset:", name, "shape:", X.shape, "===")
        Xs, scaler = standardize(X)

        # --- KMeans ---
        k_guess = 3 if name.startswith("Iris") else 4 if 'Blobs' in name else 2
        klabels, centers, kmodel = run_kmeans(Xs, k=k_guess)
        evaluate_clusters(Xs, klabels, name + " KMeans")
        # PCA for plotting
        X2, pca = pca_transform(Xs)
        centers2 = pca.transform(centers) if centers is not None else None
        plot_2d(X2, klabels, title=f"{name} - KMeans (k={k_guess})", centers2=centers2)

        # --- DBSCAN ---
        # Heuristic eps: try multiple values for robust result (could tune more rigorously)
        # For higher-dim data, eps must be chosen carefully.
        if name.startswith("Iris"):
            eps = 0.9
            min_samples = 5
        elif 'Blobs' in name:
            eps = 0.5
```

```
        min_samples = 5
    else:  # moons
        eps = 0.2
        min_samples = 5

    dlabels, db = run_dbscan(Xs, eps=eps, min_samples=min_samples)
    evaluate_clusters(Xs, dlabels, name + " DBSCAN")
    # plot
    X2, pca = pca_transform(Xs)
    plot_2d(X2, dlabels, title=f"{name} - DBSCAN (eps={eps},
min_samples={min_samples})", centers2=None, show_centers=False)

if __name__ == "__main__":
  main()
```

How to run

1. Save as clustering_lab.py.
2. Install dependencies:

```
pip install numpy matplotlib seaborn scikit-learn pandas
```

3. Run:

```
python clustering_lab.py
```

What the code does (high level)

- Loads Iris and two synthetic datasets (blobs & moons).
- Standardizes features (important for clustering).
- Runs K-Means with a guessed k (Iris: 3, Blobs: 4, Moons: 2).
- Runs DBSCAN with suggested eps/min_samples.
- Evaluates clusters with silhouette score and Davies–Bouldin index.
- Projects data to 2D via PCA for visualization; plots cluster assignments, K-Means centroids.

Parameter tuning tips

- **K-Means**: use the elbow method (plot inertia vs k) or silhouette score to choose k.
- **DBSCAN**: plot k-distance graph (k = min_samples) to choose eps. DBSCAN is sensitive to eps and min_samples; works poorly for high-dimensional sparse data unless you reduce dimensionality first.
- **Scaling**: Always standardize or normalize features before distance-based clustering.

Interpreting results

- **K-Means** works well on the blobs dataset (spherical clusters) and often recovers the Iris species reasonably, but may mix some species because Iris data is not perfectly separable in feature space.
- **DBSCAN** will mark low-density points as noise (label = -1) and can find non-spherical clusters (works well on moons if eps chosen correctly).

- **Silhouette score**: closer to +1 means better-defined clustering; negative values indicate overlapping clusters.

Example expected outputs (text)
=== Dataset: Iris (4D) shape: (150, 4) ===
Iris (4D) KMeans -> clusters: 3, silhouette: 0.4590, davies-bouldin: 0.7358
Iris (4D) DBSCAN -> clusters: 3, silhouette: 0.3521, davies-bouldin: 0.9123

=== Dataset: Blobs (2D) shape: (500, 2) ===
Blobs (2D) KMeans -> clusters: 4, silhouette: 0.78, davies-bouldin: 0.32
Blobs (2D) DBSCAN -> clusters: 4, silhouette: 0.75, davies-bouldin: 0.38

=== Dataset: Moons (2D) shape: (500, 2) ===
Moons (2D) KMeans -> clusters: 2, silhouette: 0.49, davies-bouldin: 0.85
Moons (2D) DBSCAN -> clusters: 2, silhouette: 0.61, davies-bouldin: 0.45

Questions:
1.Explain the fundamental difference between K-Means and DBSCAN. When would you prefer DBSCAN over K-Means?
2. Why is feature scaling important before applying clustering algorithms like K-Means and DBSCAN?
3.What is the role of PCA in visualizing multidimensional data, and does PCA affect the clustering results?

Experiment No 7: Descriptive Analytics & Visualization of Various Data Types

Aim: To apply descriptive analytics techniques (summary statistics, distribution analysis, correlation analysis, categorical summaries) on different data types and visualize insights using Python (Pandas, Matplotlib, Seaborn).

Dataset

You may use any of the following:

- **Iris Dataset** (numeric, categorical)
- **Titanic Dataset** (mixed types)
- **Sales Time-Series Data**
- Synthetic data (randomly generated)

In the below code, we use **Iris** + **sample categorical + time-series data**.

Descriptive Techniques Covered

A. For Numerical Data

- Mean, Median, Mode
- Min, Max
- Variance, Standard Deviation
- Percentiles
- Distribution plots (Histogram, KDE)
- Boxplots

B. For Categorical Data

- Frequency counts
- Bar plots
- Pie charts

C. For Multivariate Data

- Correlation Matrix
- Heatmap
- Scatter plot
- Pairplot

D. For Time-Series Data

- Line plots
- Trend visualization
- Rolling statistics

Full Python Code for Descriptive Analytics & Visualization

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
```

```
sns.set(style="whitegrid")

# ------------------------------
# Load IRIS dataset
# ------------------------------
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

print("Dataset Head:")
print(df.head())

# ----------------------------------
# A. NUMERICAL DESCRIPTIVE STATISTICS
# ----------------------------------
print("\nDescriptive Statistics:")
print(df.describe())

# Histogram for feature distribution
df.hist(figsize=(10,6), bins=15, edgecolor='black')
plt.suptitle("Histogram of Numerical Features")
plt.show()

# Boxplot
plt.figure(figsize=(8,6))
sns.boxplot(data=df.iloc[:, :4])
plt.title("Boxplot for Numerical Features")
plt.show()

# KDE (Density Plot)
plt.figure(figsize=(8,5))
sns.kdeplot(df['sepal length (cm)'], shade=True)
plt.title("Density Plot of Sepal Length")
plt.show()

# ----------------------------------
# B. CATEGORICAL ANALYSIS
# ----------------------------------
print("\nCategory Counts:")
print(df['species'].value_counts())

plt.figure(figsize=(6,4))
```

```
sns.countplot(x='species', data=df)
plt.title("Count Plot of Species")
plt.show()

# Pie chart
df['species'].value_counts().plot(kind='pie', autopct='%1.1f%%', figsize=(5,5))
plt.title("Species Distribution")
plt.ylabel("")
plt.show()

# -----------------------------------
# C. MULTIVARIATE ANALYSIS (CORRELATION)
# -----------------------------------
corr = df.iloc[:, :4].corr()
print("\nCorrelation Matrix:")
print(corr)

plt.figure(figsize=(7,5))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# Pairplot
sns.pairplot(df, hue='species')
plt.suptitle("Pairplot of IRIS Dataset", y=1.02)
plt.show()

# -----------------------------------
# D. TIME SERIES ANALYSIS (Synthetic Example)
# -----------------------------------
# Create a synthetic time series dataset
dates = pd.date_range(start='2024-01-01', periods=50)
sales = np.random.randint(100, 500, size=50)
time_df = pd.DataFrame({'date': dates, 'sales': sales})

plt.figure(figsize=(10,4))
plt.plot(time_df['date'], time_df['sales'], marker='o')
plt.title("Sales Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# Rolling average
time_df['rolling_mean'] = time_df['sales'].rolling(window=5).mean()

plt.figure(figsize=(10,4))
plt.plot(time_df['date'], time_df['sales'], label='Actual', marker='o')
plt.plot(time_df['date'], time_df['rolling_mean'], label='Rolling Mean (5)', linewidth=3)
plt.title("Sales Trend with Rolling Average")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Interpretation of Results

Numerical Data Insights

- Summary stats help identify central tendency & variability.
- Histogram shows how features are distributed.
- Boxplots reveal outliers.
- Correlation heatmap shows relationships (e.g., sepal length & sepal width).

Categorical Data Insights

- Count plot and pie chart show class imbalances.
- Useful for understanding dataset representation.

Multivariate Insights

- Pairplot visually shows cluster separability among species.
- Heatmap highlights correlated features (useful for feature selection).

Time-Series Insights

- Trend patterns and rolling averages help detect seasonality or fluctuations.

Conclusion

Descriptive analytics provides a foundation for understanding patterns, distributions, correlations, and trends in data. Visualization helps convert raw numbers into insights that guide decision-making and further modelling.

Questions

Q1. What is the difference between descriptive analytics and predictive analytics?

Q2. Why is normalization or standardization important before applying multivariate visualizations or clustering?

Q3. What are the advantages of using PCA for visualizing high-dimensional datasets?

Experiment No. 8: Predictive Analytics & Machine Learning Models

Aim

To develop and apply predictive analytics models using machine learning algorithms such as Linear Regression, Logistic Regression, Decision Trees, and Random Forests on real-world datasets.

Dataset

You may use:

- **Iris Dataset** (classification)
- **Titanic Dataset** (classification)
- **House Price Dataset** (regression)
- Or any real-world CSV file

Below, **Iris (classification)** + **Synthetic Regression** examples are used.

Theory

A. Predictive Analytics

Predictive analytics uses historical data + statistical algorithms + machine learning techniques to predict future outcomes.

Key Steps:

1. Data Understanding
2. Preprocessing (cleaning, encoding, scaling)
3. Feature selection
4. Model training
5. Model testing
6. Evaluation
7. Deployment

B. Machine Learning Algorithms Used

Type	Algorithm	Use Case
Regression	Linear Regression	Predict continuous values
Classification	Logistic Regression	Binary/multiclass outcomes
	Decision Tree Classifier	Rule-based predictions
	Random Forest Classifier	High-accuracy ensemble model

C. Evaluation Metrics

Regression

- MAE (Mean Absolute Error)
- MSE (Mean Squared Error)
- RMSE (Root Mean Squared Error)
- R^2 (Coefficient of Determination)

Classification

- Accuracy
- Precision, Recall, F1-score
- Confusion Matrix
- ROC Curve & AUC

Full Python Code: Predictive Analytics Pipeline

```
# predictive_analytics_lab.py
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

# ---------------------------------------
# PART 1: Classification (Iris Dataset)
# ---------------------------------------

iris = load_iris()
X = iris.data
y = iris.target

print("Dataset shape:", X.shape)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MODELS
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(max_depth=3),
```

```
    "Random Forest": RandomForestClassifier(n_estimators=100)
}

print("\n=== Classification Results (Iris Dataset) ===")
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)

    print(f"\n{name} Accuracy: {acc:.4f}")
    print(classification_report(y_test, preds))

    # Confusion Matrix
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

# --------------------------------------
# PART 2: Regression (Synthetic Example)
# --------------------------------------

np.random.seed(42)
X_reg = np.random.rand(100, 1) * 10
y_reg = 3 * X_reg.flatten() + 7 + np.random.randn(100) * 2

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_size=0.3)

model_lr = LinearRegression()
model_lr.fit(X_train_r, y_train_r)
y_pred_r = model_lr.predict(X_test_r)

print("\n=== Regression Results ===")
print("MAE:", mean_absolute_error(y_test_r, y_pred_r))
print("MSE:", mean_squared_error(y_test_r, y_pred_r))
print("RMSE:", np.sqrt(mean_squared_error(y_test_r, y_pred_r)))
print("R² Score:", r2_score(y_test_r, y_pred_r))

# Plot regression line
plt.scatter(X_test_r, y_test_r, color='blue', label='Actual')
plt.plot(X_test_r, y_pred_r, color='red', label='Predicted')
```

```
plt.title("Linear Regression Fit")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

Interpretation of Results

Classification (Iris Dataset)

- Logistic Regression = simple, high accuracy for linearly separable data.
- Decision Tree = easy to interpret but prone to overfitting.
- Random Forest = best accuracy due to ensemble learning.

Regression

- Linear Regression fits synthetic data well (R^2 close to 1 indicates good model).
- Low RMSE shows accurate predictions.

Conclusion

Predictive analytics allows forecasting and classification using machine learning models. Logistic Regression, Decision Tree, and Random Forest successfully classified the Iris dataset, while Linear Regression efficiently predicted continuous outcomes. The experiment demonstrates the end-to-end workflow of building predictive models.

Questions

Q1. What is the difference between regression and classification in machine learning?

Q2. Why do we split the dataset into training and testing sets?

Q3. Which model performed best in this assignment and why?